

NO PROMPT-FIRST COMPLETION

THREE.JS EVIDENCE GRAPH

A multi-agent production system for AAA-grade browser vertical slices without downloaded art, audio, models, textures, or asset packs — now with two determinism regimes, calibrated critics, compute economics, and a named human authority.



FIG. 0 — P95 / P99 FRAME-TIME CONVERGENCE ACROSS REPAIR CYCLES. TARGET PROFILE, ILLUSTRATIVE — NO RUN DATA EXISTS YET. SEE DOCUMENT STATUS.

OPERATIONAL MANUAL

Graph orchestration, subagent boundaries, procedural asset compilation, deterministic replays, calibrated critics, frame-time gates, provenance, rollback, economics, and convergence.

THREE.JS R185 BASELINE
WEBGPU + WEBGL 2 DECISION GATE
SOURCE-CONTROLLED GENERATION
TWO DETERMINISM REGIMES
EVIDENCE-BACKED RELEASE CRITERIA

The prompt is not the production system.

A high-capability model can write a great deal of code. It cannot prove quality merely by describing its intention, scoring its own screenshots, or repeating the same loop a fixed number of times.

The replacement architecture is a repository-local evidence graph. Deterministic code controls state, scheduling, verification, budgets, retry, rollback, and release. Models perform bounded reasoning and implementation inside that control plane.

The old playbook correctly made screenshots and metrics central. The first edition of this manual separated authority, implementation, evaluation, and release. This edition goes one step further and applies the law to itself: it opens with its own defect ledger, states exactly which reproducibility claims are physically possible, prices every critic call, and names the human who holds the constitution.

CORE EQUATION

Intent + graph + evidence + convergence

Not: intent + one giant prompt + self-reported success.
Evidence becomes enforceable rather than persuasive.

QUALITY TARGET

One narrow, highly resolved browser vertical slice with coherent gameplay, authored composition, stable frame pacing, reproducible procedural resources, and complete release evidence.

TRUST MODEL

Builder output is untrusted until separate read-only evaluators validate it against independently captured artifacts. Critic output is untrusted until the critic passes calibration.

ASSET RULE

Every visible and audible artifact is generated from source-controlled code, explicit parameters, and deterministic seeds — and the dependency graph is treated as an asset channel.

EVIDENCE CLASSES NEW

Bit-exact claims are reserved for simulation truth. GPU-rasterized artifacts are compared within declared tolerance. The regime is named per artifact, never implied.

NAVIGATION

Contents

The PDF explains the system. The repository carries the executable contracts, graph state, task packets, schemas, test harness, and evidence. Entries below are linked; the outline panel of your PDF reader mirrors them.

A	v1 → v2 defect ledger	This edition applied to itself: ten recorded corrections	04
00	Diagnosis	Research basis, failure anatomy, product target, acceptance model	05
01	Control plane	The 15-node graph, typed state, transitions, trust, task packets	10
02	Three.js architecture	Renderer proof, simulation truth, determinism regimes, performance, audio	18
03	Asset-free production	Constitution, supply chain, provenance, art grammar, authored beats	30
04	Evidence + convergence	Deterministic capture, critic calibration, repair, branching, stop conditions	37
05	Operations NEW	Compute economics, gate batteries, and the director's authority	46
06	Prompt language	Operational replacements for aesthetic wishes; the prompt as interface	49
07	Implementation program	Eight phases, provider adapters, red-team checklist, release audit	52
A	Orchestrator system prompt	Rendered from the canonical repository copy at document build	58
B	Normative schemas NEW	Task packet, defect record, run manifest — validation is code, not etiquette	62
R	Primary sources + document status	What is sourced, what is synthesis, and what does not exist yet	64

Recommended reading path

Start with the defect ledger and the target and gates, then the evidence graph, the determinism regimes, the renderer decision, the provenance constitution, critic calibration, the economics of a run, the full orchestrator prompt, and the release checklist.

APPLIED TO ITSELF

v1 → v2 defect ledger

A framework that demands defect records from workers should publish its own. Each entry below was raised in structured review of v1, accepted, and repaired in this edition.

ID	OBSERVED DEFECT IN V1	CORRECTION IN V2	WHERE
DOC-001	Asserted the primacy of evidence while shipping only prose; no executable artifact accompanied the claims.	Normative schemas ship in Appendix B; companion-repository publication is the v2.x release criterion; the status page declares what does not exist yet.	APP. B, R
DOC-002	Topology mismatch: the graph figure showed 8 nodes; the orchestrator prompt executed 15.	The 15-node graph is canonical and drawn in full; the 8-phase roadmap is labeled a condensed operator view of it.	§01, §07
DOC-003	Determinism overclaim: matching hashes were demanded of GPU-rasterized output, which is not bit-stable across devices, drivers, or browsers.	Two determinism regimes: bit-exact hashing for simulation truth; declared-tolerance comparison for rasterized and cross-profile evidence.	§02
DOC-004	Cross-browser replay identity assumed JavaScript math is portable; ECMA-262 leaves transcendental precision implementation-defined.	A deterministic math kernel is required when cross-browser identity is gated; bare Math.* narrows the claim.	§02
DOC-005	No economics: nothing priced a run, tiered the models, or said when the critic panel may be skipped.	Model-tier policy, three gate batteries, and a mandatory cost ledger in every run manifest.	§05
DOC-006	No named human. “Recorded authority change” never said by whom, recorded where, or bounded how.	The director role: four sign-off pauses, a dashboard specification, and hard limits on what a human may not do.	§05
DOC-007	Asymmetric skepticism: builders were untrusted, critics were believed on sight.	Critic calibration: known-defect pairs, admissibility thresholds, both-order review, drift re-runs on model change, schema-enforced findings.	§04
DOC-008	Audio had a provenance law but no quality gate; evidence was visual and trace-heavy.	An audio evidence gate: offline-rendered event timeline, loudness and true-peak budgets, voice limits, dropout detection.	§02
DOC-009	Supply chain unaddressed — the package graph can smuggle media past a src/ scan; accessibility was absent from an “AAA” contract.	Dependencies are an asset channel; the scanner audits built bundles; accessibility joins the product target and the release checklist.	§03, §00
DOC-010	Typographic defects: the cover bar collided with the version number; hyphens dropped from compound terms; the appendix wrapped mid-token.	The type system was rebuilt: three faces, one scale, semantic color roles, pre-wrapped appendix source, a collision-free cover grid.	GLOBAL

Convention note — document defects use the DOC- prefix; product defects in a run use VIS-, GMP-, PRF-, AUD-, PRV-, ARC- prefixes as in §04.

SECTION 00 / DIAGNOSIS

FAILURE HAS AN ANATOMY

What the previous playbook already got right, the six mechanisms that make detailed prompts produce weak games anyway, and the release contract that replaces “make it AAA” with eleven gates.



RESEARCH BASIS

Keep the evidence. Discard the self-report.

The previous playbook established three durable invariants — no external assets, no first-pass acceptance, evidence captured from the running application^[1]. Those ideas remain. This edition aligns them with Three.js r185, the WebGPU and TSL path, current browser automation, and primary research on self-correction and agent graphs^[2-19].

KEEP

Observable development harness

Stable scene states, reproducible camera positions, runtime metrics, screenshot routes, and explicit completion gates remain foundational.

DISCARD

One agent as builder, critic, scorer, approver

The same model should not implement, explain, grade, repair, and certify its own output.

KEEP

Optimize without deleting the look

Performance repairs preserve the defining art and gameplay identity through cheaper perceptual equivalents.

DISCARD

Minimum iteration count as proof

Five iterations can repeat one weak abstraction. Convergence is defined by gates, clean regression cycles, and reproducibility.

KEEP

Narrow vertical-slice scope

A short route or encounter is the correct unit for concentrated quality under procedural-only constraints.

DISCARD

Monolithic prompt as repository memory

Stable laws, current plans, architecture, art rules, and evidence belong in versioned, mechanically checked artifacts.

Research rule

Official documentation controls current APIs and product capabilities. Primary papers support research claims. Design recommendations are engineering synthesis, not product facts.

The old control model asked the prompt to behave like a studio. The new model builds a small studio around the prompt.

23

PRIMARY SOURCE
FAMILIES

06

VERIFICATION LOOPS

00

DOWNLOADED FINAL
ASSETS

02

DETERMINISM REGIMES

FAILURE ANATOMY

Why detailed prompts still produce weak games

Six mechanisms recur in prompt-first production. Each one survives good intentions, strong models, and long instructions, because none of them is a knowledge problem. They are control problems.

01

Context collision

Product goals, architecture, art direction, budgets, QA, and release instructions compete inside one context window.

02

Correlated judgment

The builder sees its own effort and intention, so defects are softened or rationalized rather than recorded ^[18].

03

Unverified prohibitions

“No external assets” is only a sentence unless network, binary, bundle, and manifest audits enforce it.

04

Randomness without grammar

More variation can increase repetition and incoherence when composition has no authored constraints.

05

Average-FPS camouflage

Average frame rate hides shader compilation stalls, long frames, garbage-collection spikes, and unstable input response.

06

No rollback boundary

Repeated edits accumulate regressions without candidate branches, protected metrics, and accepted baselines.

Design response

Convert every important instruction into one of seven machine-legible forms:

AUTHORITY

/

INVARIANT

/

INTERFACE

/

ARTIFACT

/

METRIC

/

TRANSITION PREDICATE

/

FAILURE RESPONSE

A sentence that cannot become one of these forms is decoration, and decoration does not survive contact with a build agent.

PRODUCT TARGET

Define AAA-grade as a release contract

Do not ask for an entire AAA game. Ask for a narrow browser vertical slice whose defining systems are fully resolved and whose evidence is reproducible.

DIMENSION	OPERATIONAL TARGET
Scope	One primary verb, one controlled route or encounter, one complete success, failure, and reset loop.
Duration	45–90 seconds of authored progression, not an open procedural sandbox.
Presentation	Five to seven deliberate visual and gameplay beats with stable evidence states.
Art	One coherent procedural grammar with bounded variation and causal material logic.
Feel	Input, camera, animation, FX, lighting, and audio respond to the same gameplay events.
Performance	Declared frame-time budgets pass on named device profiles; no identity deletion.
Access NEW	Reduced-motion preference honored; route-critical signaling survives grayscale and color-vision-deficiency simulation; HUD text contrast at or above 4.5:1 [22, 23] .
Provenance	All final media is generated locally from source and seed; network and bundle audits pass.
Release	Clean checkout reproduces generators, replays, screenshots, traces, and hashes.

START HERE

Concept fit

Hard-surface vehicles, mechanical cities, abstract biomes, stylized architecture, designed weather systems, and signal-rich interfaces fit the constraint.

HIGH RISK

High-risk first targets

Photoreal human faces, hair, cloth, crowds, natural performance, and dense character animation are inefficient foundations for a zero-asset first vertical slice.

The Access dimension is part of the product, not a compliance afterthought: a slice whose route reads only through hue has an art-direction defect.

ACCEPTANCE MODEL

Separate hard gates from aesthetic judgment

Eleven hard gates, each with machine-verifiable evidence and a release predicate. Two are new in this edition; two carry regime labels from §02.

GATE	MACHINE-VERIFIABLE EVIDENCE	RELEASE PREDICATE
Build	Typecheck, lint, unit, structural tests, production build	All pass from clean checkout
Runtime	Console, renderer and device errors, unhandled rejections	No release-blocking event
Determinism · A	Input replay, seed manifest, periodic simulation-state hashes	Hashes match accepted baseline, Regime A
Gameplay	Completion, collision, crash, reset, checkpoint tests	Required states reachable and recoverable
Performance	Median, p95, p99, 1% low, long frames, CPU and GPU trace	Profile-specific budgets pass
Audio NEW	Offline-rendered event timeline: loudness, true peak, voice count, dropouts	Declared audio budgets pass, §02
Provenance	Network log, file and built-bundle scan, generator manifest, output hashes	No unapproved media or request
Regression · B	Golden screenshots, state snapshots, protected metrics	No unauthorized delta within declared tolerance
Compatibility	Playwright projects and named browser and device matrix	Required profiles pass
Accessibility NEW	Reduced-motion run, CVD-simulated route captures, HUD contrast report	Access targets in §00 pass
Reproducibility	Regeneration command and checksum comparison	Outputs reproduce within declared regime

Aesthetic evaluation

Use anchored criteria, pairwise comparison, and independent, calibrated visual critics (§04). Do not turn taste into a false decimal such as 4.4/5.

Release rule

Aesthetic approval never overrides a hard gate. A hard gate never proves art direction. Both systems must pass.

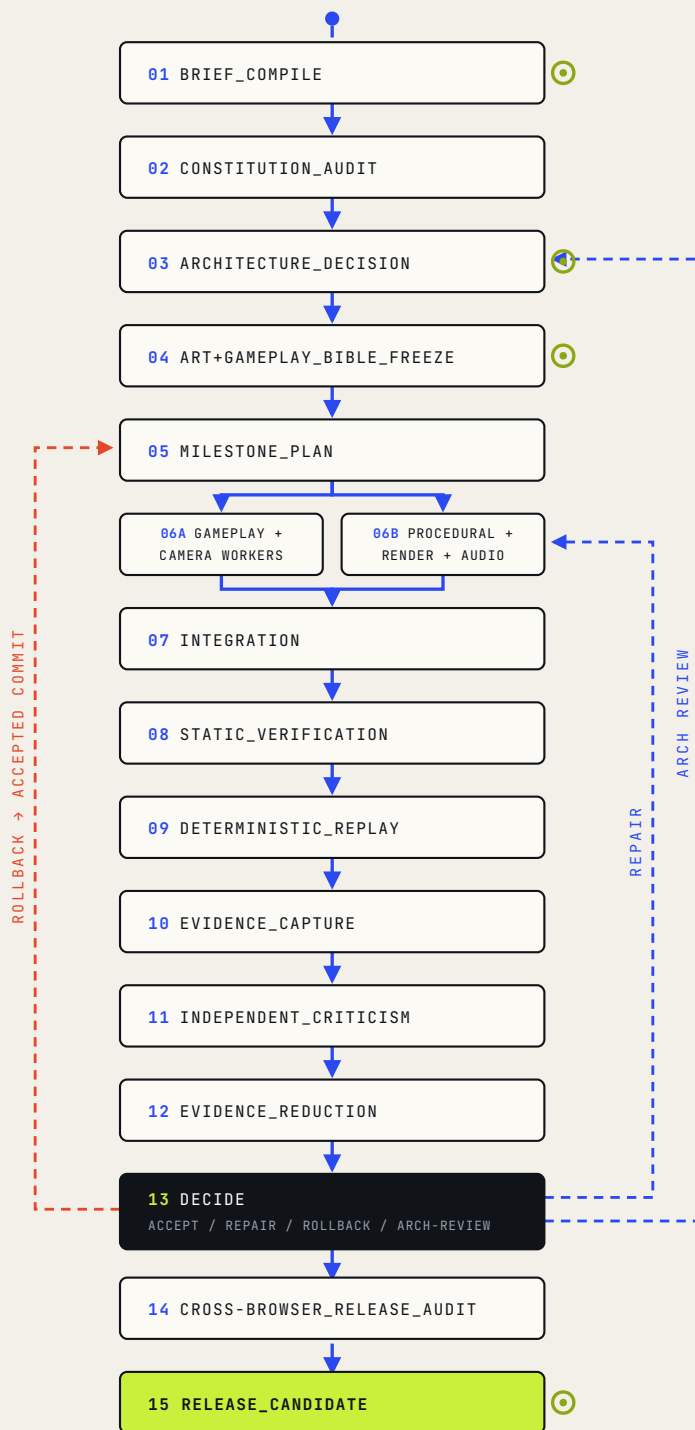
SECTION 01 / CONTROL PLANE

ONE CONTROLLER, BOUNDED SPECIALISTS

The production run is a typed graph executed by deterministic code. Models reason inside nodes; they never own scheduling, budgets, retries, or state truth. This section defines the canonical 15-node topology, its state, its edges, and its trust boundaries.

EXECUTABLE TOPOLOGY

The evidence graph, in full



Control rule

Deterministic code owns graph transitions. A model may recommend a transition but cannot bypass its predicate.

- 01 Compile intent into a testable product contract.
- 02 Freeze constitution, art bible, architecture, and gates.
- 03 Parallelize only disjoint ownership or read-only work.
- 04 Capture screenshots, traces, hashes, logs, audio renders, and network evidence independently.
- 05 Repair root causes, compare candidates, roll back regressions, escalate plateaus.

Legend

Blue solid — forward edge. **Blue dashed** — repair and architecture-review loops.

Red dashed — rollback to the accepted commit.

Ring mark — director sign-off pause (§05): contract, renderer record, freeze, release.

Resolves DOC-002: this 15-node graph is canonical. The eight-phase roadmap in §07 is a condensed operator view of the same topology, not a second topology.

TYPED STATE

Conversation history is not project state

```
type GraphState = {
  projectId: string
  phase: Phase
  authorityVersion: string
  acceptedCommit: string | null
  activeTasks: TaskRef[]
  blockedBy: IssueRef[]
  evidenceRun: string | null
  gates: Record<GateId, GateResult>
  defects: DefectRecord[]
  protectedMetrics: MetricSnapshot
  provenanceDigest: string | null
  budgets: { tokens: number; wallTimeMs: number; retries:
number }
  costLedger: CostEntry[] // NEW - see §05, Appendix B
  history: TransitionRecord[]
}
```

State must answer

Which authority version controls this run? What commit is accepted? Which gates failed? Which metrics are protected? What work is safe in parallel? What budget remains — and what has already been spent, by which role?

The orchestrator persists state after every node. A run resumes without asking a model to reconstruct what happened from prose.

IMMUTABLE

Run ID, seed, commit, browser profile, viewport, renderer backend, authority digest.

MUTABLE

Task status, active defects, retry count, candidate branches, evidence index, cost ledger.

NEVER INFERRED

Pass/fail state, artifact existence, asset provenance, benchmark result, release status.

TRANSITION ENGINEERING

Edges need predicates, not encouragement

STATIC VERIFICATION → REPLAY → EVIDENCE → CRITICS → DECIDE

FORWARD**Forward transition**

Predicate: build passes, no structural violation, required fixtures exist, task commit is clean. Failure returns the exact command, stderr, affected files, and remaining retry budget.

REPAIR**Repair transition**

Predicate: a gate fails and the defect record has evidence, hypothesis, protected metrics, and an acceptance test. Generic criticism is rejected by the schema validator, not by etiquette.

ROLLBACK**Rollback transition**

Predicate: no target improvement, any hard-gate regression, or insufficient evidence confidence. Restore the accepted commit; the failed candidate branch remains for forensics.

RELEASE**Release transition**

Predicate: all eleven gates pass, no critical or major defect remains, two regression cycles are clean, provenance passes, and a clean checkout reproduces artifacts within the declared regime.

A graph edge is a decision boundary. It should be executable, observable, and reversible — and every traversal is appended to history[] with its evidence run ID, so the run can be audited without interviewing anyone.

TRUST BOUNDARIES

Separate intention from proof

MAY EDIT

Builder

Changes only assigned implementation files, runs local acceptance, and returns artifacts.

CAPTURES

Harness

Controls seed, tick, browser, viewport, replay, screenshots, traces, logs, audio renders, and network interception.

MAY REJECT

Critic

Reads evidence and contracts, identifies observable defects, and returns structured findings without editing code.

Why independence matters

Research on intrinsic self-correction shows that a model can fail to improve, or degrade, when asked to correct itself without reliable external feedback^[18]. Self-reflection can generate hypotheses; it should not own release authority. Multi-agent judging work shows structured, role-separated review outperforms one model grading itself^[19,24].

The same skepticism now runs in both directions: builders are untrusted until evidence passes, and **critics are untrusted until calibration passes**. A critic model earns authority the same way a build does — against known cases, under blind protocol. The calibration procedure is defined in §04 and resolves DOC-007.

Forbidden authority combinations

- Builder + release approver
- Performance optimizer + protected-feature waiver
- Critic + implementation edits
- Orchestrator + evidence fabrication
- Candidate owner + blind-label decoder
- Critic + own calibration grader

NEW

SUBAGENT DESIGN

Roles, authority, and write boundaries

AGENT	AUTHORITY	WRITE SCOPE	CANNOT DO
Orchestrator	Graph state, budgets, transitions	Task manifests and scheduling state	Bulk implementation or self-certification
Brief compiler	Product contract	PRODUCT_CONTRACT.md	Choose implementation details
Architect	Renderer, layers, dependencies	Architecture and decisions	Silently override constitution
Gameplay worker	Simulation, controls, camera, collision	Assigned gameplay modules	Edit global renderer config
Procedural worker	Generators, grammar, LOD, manifests	Procedural modules	Import opaque resources
Rendering worker	Materials, TSL, lighting, post	Rendering modules	Change gameplay truth
Audio worker NEW	DSP graphs, mixes, event bindings	Audio modules	Alter gameplay event truth
QA worker	Replay, screenshots, instrumentation	Tests and harness	Approve its own harness defect
Performance worker	Profiling and measured optimization	Isolated candidate	Delete protected identity
Visual critic	Composition and art direction	Read-only defect records	See builder rationale in blind review
Release auditor	Final evidence and provenance	Release manifest only	Waive a failed gate

Agent-count rule

Use the fewest roles that create independent expertise, safe parallelism, and independent verification. More agents are not automatically better — each role is also a line in the cost ledger (\$05).

DELEGATION CONTRACT

Every worker receives a bounded task packet

```
{
  "task_id": "RND-042",
  "objective": "Reduce first-use shader stutter in hero route",
  "context_paths": ["ARCHITECTURE.md", "src/rendering/"],
  "allowed_files": ["src/rendering/preview.ts", "tests/perf/"],
  "forbidden_files": ["PRODUCT_CONTRACT.md", "src/gameplay/"],
  "invariants": ["hero material identity", "no network media"],
  "acceptance_commands": ["npm run test:perf -- hero-preview"],
  "expected_evidence": ["trace.json", "before-after.csv"],
  "budget": {"wall_time_min": 35, "retries": 2,
    "model_tier": "mid"},
  "failure_statuses": ["BLOCKED", "NO_IMPROVEMENT",
    "REGRESSION"]
}
```

- One objective
- Known context paths
- Explicit file ownership
- Protected invariants
- Executable acceptance
- Expected artifacts
- Budget, retry limit, and model tier **NEW**

The packet validates against `task-packet.schema.json` — Appendix B. A packet that fails validation is never dispatched.

A valid return

A status from the declared set, artifacts at the expected paths, the acceptance command output, and a diff confined to allowed files.

An invalid return

A prose success claim, an edit outside the allowed set, a missing artifact, or an undeclared status. Rejected by code before any model reads it.

REPOSITORY CONTROL PLANE

Give agents a map, not a manual

Repository-local, versioned artifacts are the system of record. Current harness-engineering practice reports that a monolithic instruction file crowds out task context, decays, and is difficult to verify^[2].

```

/
├── AGENTS.md                # short map
├── CONSTITUTION.md          # non-negotiable laws
├── PRODUCT_CONTRACT.md      # scope and acceptance
├── ART_BIBLE.md             # grammar and composition
├── ARCHITECTURE.md          # renderer and layers
├── QUALITY_GATES.md         # metrics and predicates
├── WORKFLOW.md              # graph policy
├── DECISIONS.md             # approved authority changes
├── PROVENANCE.json          # generator registry
├── orchestration/{graph,state,transitions,reducers,adapters}/
├── orchestration/prompts/    # canonical Appendix A lives here
├── agents/                  # role cards
├── tasks/{active,completed}/ # typed packets
├── docs/exec-plans/{active,completed}/
├── src/
├── {core,simulation,gameplay,procedural,rendering,audio,presentation,qa}/
├── tests/{unit,structural,replay,visual,performance,provenance}/
├── tests/critics/calibration/ # known-defect pairs, §04
├── tools/scanners/          # bundle + network provenance
├── evidence/run-0001/
└── {manifest,screenshots,traces,metrics,logs,audio,provenance}/

```

AGENTS.md

A concise navigation map, authority order, commands, and links. Not the encyclopedia.

Mechanical enforcement

Broken-link checks, contract ownership, freshness checks, layer-import lints, evidence schema validation, stale-plan audits, and unowned-debt reports.

Three paths are new in v2: the canonical orchestrator prompt, the critic calibration library, and the bundle scanners. Each exists because a v1 claim could not be enforced without it.

SECTION 02 / THREE.JS ARCHITECTURE

PROVE THE RENDERER. FIX THE CLOCK.

Pin r185, decide WebGPU against WebGL 2 with representative proof, keep gameplay truth on a fixed tick — and say honestly which artifacts can be bit-exact and which can only be equivalent within tolerance. Then measure the audio the same way as the pixels.

2026 BASELINE

Pin Three.js. Prove the renderer.

Three.js r185 is the release baseline for this edition^[8]. Pin the exact version in the lockfile and record the renderer decision in architecture documentation.

TRACK	STRENGTH	CONSTRAINT
WebGPURenderer + TSL	WebGPU primary, WebGL 2 fallback, node materials, TSL, node post stack, MRT.	Experimental; legacy ShaderMaterial, onBeforeCompile, and EffectComposer workflows require migration.
WebGLRenderer	Mature pure-WebGL 2 path and compatibility with established patterns.	Not the focus for major new features; dual pipelines increase maintenance cost.

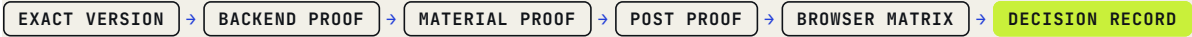
DO NOT CHOOSE BY TREND

Decision gate

Choose from the product support matrix, representative material proof, post-processing proof, frame-time trace, browser coverage, and missing-feature log.

Pipeline rule

Avoid maintaining two complete material pipelines unless the support contract makes it unavoidable.



ARCHITECTURE PROOF

The renderer decision is a milestone, not a preference

Six steps, each producing an artifact. The decision is frozen only when all six exist in the evidence run.

- 01 Representative material**
Build the hardest defining material: wet coated metal, layered emissive glass, terrain blend, or another signature surface.
- 02 Representative post stack**
Test only effects required by the art bible. TSL exposes bloom, motion blur, SSR, SSGI, GTAO, DoF, FXAA, SMAA, film, LUT, and related utilities^[10].
- 03 Representative stress scene**
Use target transparency, instance count, lights, particles, route depth, and dynamic state.
- 04 Browser and device matrix**
Run declared profiles, including forced WebGL fallback where applicable.
- 05 Trace and capability report**
Record frame-time distributions, compilation stalls, memory, unsupported requirements, and visual deltas.
- 06 Freeze the decision**
Write the accepted track and the rejected alternative into ARCHITECTURE.md — a director sign-off point (\$05).

SIMULATION TRUTH

Rendering is not the authoritative clock

INPUT FRAME → FIXED TICK → STATE HASH

Use a fixed simulation step, render interpolation, named deterministic random streams, and recorded inputs. Presentation systems may use a separate clock, but gameplay truth must remain reproducible.

Three.js timing

Three.js documents `Timer` as the alternative designed to avoid conceptual problems in `Clock`; `Clock` is deprecated since r183^[11,12]. `Timer` can support presentation timing; the fixed gameplay step remains your architecture.

Randomness rule

Never use `Math.random` for authoritative gameplay or world generation. A random stream is named, seeded, owned, and logged.

COMPLETE STATE

Replay identity

```
scene=hero
seed=481516
tick=2160
quality=high
renderer=webgpu
browser=chromium
viewport=1440x900
commit=7a19d2f
authority=sha256: ...
mathkernel=v1 // NEW
```

Two determinism regimes

Bit-exact is a property of simulation. Tolerance is a property of pixels. v1 demanded matching hashes from GPU output; that claim is not physically available, and pretending otherwise trains agents to launder failures. Every artifact now declares its regime.

REGIME A – BIT-EXACT

Simulation state hashes, input replays, seed manifests, generator geometry and DataTexture buffers, and offline audio PCM on a pinned profile. Comparator: cryptographic hash. One bit of drift is a failure.

REGIME B – DECLARED TOLERANCE

Everything a GPU rasterizes: screenshots, diagnostic views, and any cross-profile comparison. Comparator: perceptual diff within per-view thresholds against per-profile baselines captured in a pinned environment^[15].

Why GPU output is Regime B

Rasterization order, floating-point contraction, MSAA sample positions, texture filtering, and driver optimization all vary by vendor, driver, OS, and browser version. Identical source can differ per-pixel across machines. Demand equivalence, not identity.

Why bare Math.* breaks Regime A

ECMA-262 leaves transcendental functions implementation-approximated: sin, cos, pow, exp, and log may differ in final bits across engines^[20]. When cross-browser replay identity is gated, authoritative simulation uses a repository-owned deterministic math kernel; otherwise the claim narrows to same-engine, same-version.

ARTIFACT CLASS	REGIME	COMPARATOR	TOLERANCE OWNER
Simulation state hash, input replay, seeds	A	SHA-256 equality	None — exact by definition
Generator buffers: geometry, DataTexture, PCM (pinned profile)	A	Content hash	None — exact by definition
Screenshots, same pinned profile	B	Pixel diff, per-view %	QUALITY_GATES.md per view
Screenshots and text regions, cross-profile	B	Perceptual similarity + masks	QUALITY_GATES.md per profile pair
Audio, cross-profile	B	Loudness + spectral tolerance	QUALITY_GATES.md audio section

Laundering rule. A tolerance is graph state. Widening one to make a diff pass without a recorded decision is the anti-pattern named in §07, and the diff tool reads thresholds only from QUALITY_GATES.md — never from a task packet. UI text rendered from system fonts is compared per profile or masked; it is never an excuse to widen a scene threshold.

GAMEPLAY ARCHITECTURE

Bounded simulation, explicit states, simple collision truth

SYSTEM	RECOMMENDED FORM
Movement	Arcade controller with explicit acceleration, steering, damping, bank, hover, and recovery rules.
World collision	Capsule, sphere, and AABB queries, rays, swept tests, and static-world spatial partitioning.
Three.js addon	Octree can partition a static world and test capsules and other colliders against scene triangles ^[14] .
Critical states	Route, boost, near-miss, crash, reset, checkpoint, finish, pause, evidence-freeze.
Event system	One causal event stream drives camera, animation, FX, lighting, audio, telemetry, and replay.

DECISION

Framework threshold

Do not introduce an ECS, general physics engine, or elaborate framework because it appears advanced. Introduce it only when multiplicity and query patterns justify the complexity.

AVOID

Common failure

Camera, FX, and audio infer game state separately. Their timing diverges, replay becomes unstable, and feel becomes hard to debug.

RENDERING STACK

Post-processing is a budgeted composition tool

TSL path

WebGPURenderer uses node materials and TSL, which can generate WGSL or GLSL depending on backend. Its node post stack supports MRT and modern effects [\[9,10\]](#).

- Expose effect cost per quality tier
- Drive intensity from gameplay state
- Provide bloom, depth, normal, and velocity diagnostics
- Declare temporal history reset conditions

Order of operations

A post stack cannot rescue weak silhouette, composition, material separation, or route readability. Diagnose those first.

Art-direction rule

Every effect needs a visual job, activation condition, maximum intensity, performance budget, diagnostic view, and readability constraint.

Examples

Bloom separates emissive hierarchy. Motion blur communicates event-bounded speed. Chromatic aberration marks damage or boost states, never a permanent film over the frame.

PROCEDURAL TOOLKIT

Build a compiler above the primitives

Shape + Extrude

Profiles, panels, trims, signage, hard-surface components, architectural sections.

TubeGeometry

Rails, conduits, cables, trails, structural curves, pipes, track language.

CanvasTexture

Sign atlases, decals, masks, labels, generated UI surfaces, authored graphic motifs.

DataTexture

Raw-buffer noise, lookup fields, masks, height and roughness data, simulation outputs.

ART GRAMMAR → PARAMETERS + SEED → GENERATOR → REJECTION TESTS

LOD + COLLIDER → PROVENANCE → ASSET REGISTRY → RUNTIME

RULE**Important distinction**

Procedural does not have to mean generated at runtime. Build-time generation is permitted when source, seed, parameters, and output hash are reproducible.

Hero rule

The defining hero asset is assembled through authored grammar and deliberate asymmetry. Random detail remains subordinate to silhouette, proportion, and functional story.

PERFORMANCE ARCHITECTURE

Choose the batching primitive by repetition pattern

SAME GEOMETRY**InstancedMesh**

Same geometry and material, different transforms. Use for repeated windows, lights, bolts, rain streaks, lane markers, debris classes, or traffic silhouettes. It reduces draw calls.

DIFFERENT GEOMETRY**BatchedMesh**

Same material, different geometry or transforms. Use when repeated objects do not share geometry. It also targets draw-call reduction.

Prewarm

Use `renderer.compileAsync()` after representative scene construction to reduce first-use shader compilation stutter^[13].

Pool

Recycle particles, decals, trail segments, audio voices, traffic entities, route chunks, and temporary render targets.

Bound

Set maximum transparent coverage, active lights, post resolution, effect emitters, and generator output per quality tier.

MATERIAL REUSE / TEXTURE ATLASES / FRUSTUM CULLING / DISTANCE CULLING / LOD / SEGMENT RECYCLING /
ADAPTIVE DPR

Optimization rule

Optimization is not a deletion contest. Replace an expensive mechanism with a cheaper perceptual equivalent and verify protected visual evidence.

FRAME PACING

Average FPS is not a performance report

Collect

- Median, p95, p99 frame time
- 1% low and long-frame count
- Simulation CPU time; GPU time where supported
- Shader compilation stalls
- Heap and allocation growth
- Geometry and texture memory
- Draw calls, compute calls, triangles
- Input-to-render delay

Budget frame-time distributions, not a single FPS number. A 60 FPS average can coexist with visible stutter.

REQUIRED**Profile identity**

Name device class, browser, power mode, viewport, DPR, quality tier, renderer backend, seed, route, and capture duration.

Renderer visibility

Three.js `renderer.info` exposes rendering and memory statistics; `compileAsync` exists specifically to precompile materials and mitigate shader compilation stutter^[13].

ADAPTIVE QUALITY

Degrade cost before identity

SYSTEM	HIGH	MEDIUM	LOW	PROTECTED INVARIANT
Resolution	Target DPR cap	Reduced DPR	Adaptive internal scale	UI and route legibility
Post	Full required stack	Lower-res expensive passes	Cheaper substitute	Emissive hierarchy and tone
Weather	Near, mid, far layers	Reduced far density	Dense camera-local layer	Weather remains a hero feature
World	Full depth layers	Earlier LOD and fog	More impostors	Scale and route silhouette
Traffic	Close, mid, far system	Reduced distant cadence	Billboarded far traffic	Scene remains alive
Lighting	Hero real lights + fakes	Fewer real lights	Mostly emissive and fake cards	Hero integration and contrast

CONTROL

Adaptive controller

Change one cost class at a time, apply hysteresis, log every tier transition, and never adapt during deterministic evidence captures.

REJECT

Forbidden adaptation

Removing all weather, traffic, fog, wetness, post, or signature trails without a perceptual replacement.

GENERATED AUDIO

Audio obeys the same laws as the pixels

Web Audio provides oscillators, filters, envelopes, gains, delays, waveshaping, analysis, spatial nodes, and custom AudioWorklet processing. AudioWorklet runs custom processing in a separate audio thread; PannerNode represents a sound source in 3D space^[17].

Generate

Engine layers, wind, rain, boost sweeps, collision transients, UI tones, environmental drones, alarms, and procedural reverb.

Control

Voice limits, envelope pools, event ownership, spatial position, distance rolloff, directional cones, and deterministic parameter seeds.

AUDIO EVIDENCE GATE

NEW

Render the authoritative event timeline offline with OfflineAudioContext and gate the result like any other artifact: PCM content hash on the pinned profile (Regime A); integrated loudness within the declared target ± 1 LU and true peak at or below -1.0 dBTP^[21]; concurrent voice count at or below budget; zero dropout or hard-clipping events; event-to-audio offsets within declared milliseconds. Cross-profile comparison is Regime B: loudness and spectral tolerance, not sample identity.

ONE EVENT

Causal mix

Gameplay emits one event. Camera, visual FX, light response, controller feedback, and audio subscribe with declared timing. Do not infer intensity separately in each subsystem.

Deployment note

AudioWorklet custom processor modules require a secure context in normal browser deployment. Include this in local and CI serving.

Resolves DOC-008: v1 gave audio a provenance law but no quality gate. A mix that clips, drops voices, or drifts from its events now fails a hard gate, not a vibe check.

SECTION 03 / ASSET-FREE PRODUCTION

RANDOMNESS FILLS A GRAMMAR

Twelve constitutional laws, a chain of custody for every generated byte, a compiler above the geometry primitives, and quality control that catches repetition before the audience does. The dependency graph is now inside the law, not beside it.

NON-NEGOTIABLE LAW

The no-cheating constitution

- 01 No runtime or build-time request retrieves visual, geometric, typographic, animation, or audio media.
- 02 No hidden media inside base64 strings, archives, generated arrays, renamed binaries, or opaque blobs.
- 03 Every visible or audible resource is generated by deterministic source-controlled code.
- 04 CI intercepts browser requests and fails on any unapproved request after the local bundle loads.
- 05 A clean checkout regenerates resources and reproduces declared hashes or tolerances.
- 06 No GLTF, GLB, FBX, OBJ, STL, image textures, HDRIs, sprites, fonts, prerecorded audio, stock imagery, or asset packs.
- 07 Three.js, official addons, browser APIs, build tools, tests, and profilers are allowed because they are not final art assets.
- 08 Every generator records name, version, source commit, seed, parameters, output hash, owner, and regeneration command.
- 09 CI scans forbidden extensions, oversized encoded strings, opaque binaries, and unmatched generated outputs.
- 10 No implementation agent can waive the constitution. Exceptions require a recorded authority change by the director (\$05).
- 11 **The dependency graph is an asset channel.** Installs are lockfile-pinned and build-time only; packages that ship media are forbidden or stripped; the provenance scanner runs on the built bundle, not only on src/; the runtime origin allowlist is empty beyond the local bundle. **NEW**
- 12 **No font files.** UI text uses a pinned per-profile system stack or generated glyph atlases; text-bearing capture regions are compared per profile under Regime B. **NEW**

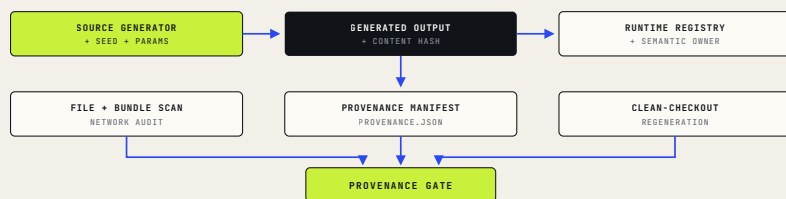
Operational wording

Every visible and audible artifact must be reproducibly generated from source-controlled code, explicit parameters, and deterministic seeds, and must pass provenance verification — including the built bundle.

Laws 11 and 12 resolve DOC-009: a package that quietly ships a texture atlas or a woff file defeats a src/ scan. Scan what ships, not what was written.

CHAIN OF CUSTODY

Provenance is generated with the asset



Manifest fields

Resource ID and semantic role; generator module and version; source commit; seed and parameters; output path and hash; build command; runtime owner; declared tolerance and regime.

GATE

Fail closed

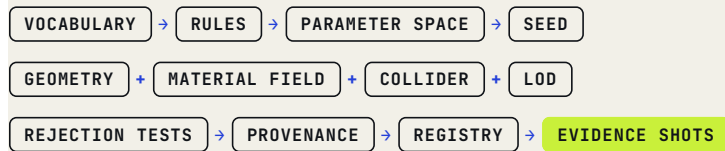
An output without a matching generator record is untrusted. An unexpected request is a failed gate, not a warning.

The bundle scanner and network audit run in the same battery as the file scan (\$05): no model calls, deterministic verdicts, every commit.

GENERATOR ARCHITECTURE

The procedural asset compiler

Treat generation as compilation: a fixed grammar, an explicit parameter space, a seed, and a battery of rejection tests between the generator and the registry.



Geometry tests

Self-intersection, degenerate triangles, invalid bounds, thinness, collider mismatch, silhouette repetition.

Material tests

Value and roughness range, emissive budget, tiling repetition, causal wear, texture memory.

Composition tests

Route occlusion, focal collision, frequency crowding, scale ambiguity, repeated motif adjacency.

Every generator is a build target. It runs headless, it is unit tested, its outputs are hashed under Regime A, and its diagnostic thumbnails join the evidence run. A generator that only works inside the game loop is a liability, not an asset pipeline.

ART SYSTEM

Randomness fills a grammar. It does not invent one.

LAYER	DECLARE	VARY	REJECT
Silhouette	Primary proportions and readable identity	Secondary breaks, asymmetry, attachments	Loss of identity at thumbnail scale
Construction	Attachment rules and functional hierarchy	Module count, spacing, orientation	Floating or implausible parts
Material	Families, value ranges, roughness logic	Wear masks, tint, local response	Flat reading or uncontrolled emissive
Frequency	Macro, medium, micro allocation	Local detail density	Uniform noise or focal competition
World	Adjacency and route-facing rules	Facade modules and skyline rhythm	Repetition clusters or route blockage
Weather	Direction, scale layers, causal surfaces	Density and gust phase	Readability loss or screen-space blanket

FIXED

Authored invariants

Silhouette, proportion, focal hierarchy, palette structure, route function, and narrative beat are authored.

Procedural freedoms

Secondary module selection, wear, local variation, non-critical placement, phase, and detail density are seeded within bounds.

AUTHORED PROGRESSION

Build a route from beats, not random segments

BEAT	NAME	FUNCTION
A	Identity reveal	Establish hero silhouette, control response, environmental rule, and dominant depth cue.
B	Compression	Narrow space, close passes, contrast suppression, or mechanical framing raises perceived speed and tension.
C	Scale release	Open vertical or lateral depth demonstrates world scale and resets composition.
D	System proof	A near miss, boost, traversal shift, or weather escalation proves coordinated feedback.
E	Material proof	A lighting state exposes signature material behavior instead of hiding it.
F	Failure and recovery	Readable failure, deterministic reset, immediate return to control.
G	Completion image	A resolved final composition and state transition that can be captured and compared.

Beat rule

A generated segment may fill a beat. It may not replace the beat’s function. Each beat is also an evidence state: a named tick, a named camera, and a golden capture in the regression set.

PROCEDURAL QUALITY CONTROL

Detect repetition before the audience does

Structural detectors

- Consecutive module signature distance
- Silhouette histogram similarity
- Facade rhythm autocorrelation
- Material palette collision
- Attachment pattern frequency
- Near-route motif reuse window
- Seed-stream correlation

Visual detectors

- Perceptual image similarity
- Edge density and orientation map
- Value-frequency overlap behind hero
- Depth-layer occupancy
- Thumbnail focal test
- Grayscale route test — extended with CVD-simulated route captures **NEW**
- Bloom-mask occupancy

Rejection sampling

Generate candidate → run structural checks → render diagnostic thumbnail → reject or accept → record seed. Do not repair every rejected candidate manually.

The grayscale route test already enforced value separation; the CVD extension renders the same captures under deutan and protan simulation so route-critical signaling never depends on hue alone^[22].

SECTION 04 / EVIDENCE + CONVERGENCE

JUDGE THE WORK. THEN JUDGE THE JUDGES.

Deterministic capture, two visual truth systems, bounded critics — and, new in this edition, a calibration protocol that decides whether a critic's findings are admissible at all. Repair root causes, branch uncertain hypotheses, and stop on evidence, not on iteration count.

DETERMINISTIC CAPTURE

Evidence is a complete state, not a screenshot name

```
?scene=hero
&seed=481516
&tick=2160
&quality=high
&renderer=webgpu
&browser=chromium
&viewport=1440x900
&ui=0
&commit=7a19d2f
```

- 01 Load the local build with network interception active.
- 02 Set seed, profile, renderer, viewport, and input replay.
- 03 Advance an exact number of fixed simulation ticks.
- 04 Wait for generator, shader, and audio readiness.
- 05 Freeze or deterministically advance presentation systems.
- 06 Record state hash, console, network, metrics, trace, screenshot, and offline audio render.
- 07 Write one immutable run manifest — validated against Appendix B.

Required views

Beauty, grayscale, depth, normals, bloom and emissive mask, velocity, route overlay, performance overlay — plus CVD-simulated route views for the accessibility gate.

INVARIANT**Capture rule**

No adaptive quality changes, random time, pointer-lock dependency, or human steering during a release evidence run.

TWO VISUAL TRUTH SYSTEMS

Pixel regression and visual criticism solve different problems

DETECT CHANGE**Pixel regression — Regime B**

Playwright can generate and compare reference screenshots. Rendering varies by OS, browser version, hardware, settings, and environment, so baselines are captured per pinned profile and compared within declared per-view tolerance^[15]. Detects missing effects, geometry, layout drift, and protected-shot changes.

JUDGE QUALITY**Semantic criticism**

A read-only, calibrated critic evaluates composition, silhouette, material separation, route readability, scale, repetition, causal detail, and art-bible alignment. Its findings are structured records, not prose reviews.

Release rule

A pixel match can preserve a bad image. A visual critic can approve an accidental regression. Release requires both.

v1 said “stable environment” and then asked for matching hashes anyway. The regime table in §02 replaces that contradiction: the comparator and its tolerance are properties of the artifact class, recorded in `QUALITY_GATES.md`.

INDEPENDENT EVALUATION

Give each critic only the evidence it needs

CRITIC	RECEIVES	MUST RETURN	DOES NOT RECEIVE
Visual	Beauty + diagnostics + art bible	Observable defect, location, severity, evidence	Builder explanation or effort history
Gameplay	Replay, input trace, event timeline, camera trace	State and feel defect and reproduction window	Visual scoring narrative
Performance	Frame trace, CPU and GPU timings, memory, profile	Dominant measured cost and confidence	Aesthetic justification
Audio NEW	Offline render, loudness report, event timeline	Mix defect with time locus and measurement	Visual evidence
Provenance	Network log, file and bundle scan, manifest, regeneration	Pass or fail with unmatched resource	Creative intent
Architecture	Diff, module graph, contracts, tests	Boundary and regression risk and action	Candidate identity in blind review

SPECIFIC

Valid criticism

“At ticks 2010–2240, the vehicle contour merges with the sign field; the bloom mask occupies the same value and frequency band behind the hero.”

REJECT

Invalid criticism

“Make it more cinematic. Add polish. Improve lighting.” Any note without artifact, location, observable failure, and acceptance condition — rejected by the schema validator before anyone reads it.

CRITIC VALIDITY

NEW

Calibrate the judges

v1 built independence and forgot validity: builders were untrusted, critics were believed on sight. A critic is a measurement instrument, and an instrument that has never seen a known input has no error bars. Four mechanisms close the gap.

01

Both orders, every pairwise

Model judges exhibit position bias. Every A/B comparison runs in both presentation orders; a preference counts only when it is stable across orders, and an unstable preference is recorded as a tie^[24].

02

Known-defect calibration set

tests/critics/calibration/ holds evidence pairs with seeded, known verdicts: a deleted bloom pass, a silhouette merge, a palette collision, a missing weather layer. Ground truth is manufactured, so accuracy is measurable.

03

Admissibility threshold + drift

A critic configuration — model, version, prompt — must meet the declared calibration accuracy (for example $\geq 90\%$) before its findings are admissible, and the set is re-run on any model or version change. Yesterday's calibration does not transfer.

04

Agreement statistics

Inter-critic agreement is reported per criterion. Persistent disagreement on one criterion means the criterion is under-specified: route it to art-bible review. Never resolve it by averaging.

A critic that has not passed calibration has no findings — it has opinions. The orchestrator refuses to admit records from an uncalibrated configuration, the same way it refuses a task packet that fails schema validation.

Resolves DOC-007. Calibration runs are cheap relative to a wasted repair cycle chasing a hallucinated defect — the economics are in §05.

ROOT-CAUSE REPAIR

Repair defects, not low scores

```
{
  "issue_id": "VIS-014",
  "evidence": ["run-042/hero.png", "run-042/bloom-mask.png"],
  "observed_failure": "Hero silhouette merges with sign field at ticks 2010-2240.",
  "root_cause_hypothesis": "Background emissive frequency and value overlap the contour.",
  "proposed_intervention": "Reduce sign occupancy; add bounded value separation zone.",
  "expected_delta": "Contour reads at 25% scale without reducing city density.",
  "protected_metrics": ["p95_frame_ms", "route_readability", "sign_density"],
  "acceptance_test": "VIS-014.spec.ts",
  "rollback_condition": "Any hard-gate regression or no pairwise preference."
}
```

SELECTION

Priority model

$$\text{priority} = \text{expected quality gain} \times \text{evidence confidence} / (\text{cost} \times \text{regression risk})$$

Also elevate any regression blocker that prevents safe evaluation. Do not select the next task by whichever category received the smallest subjective number.

- Same seed and capture state
- Target criterion improves
- Protected metrics do not regress
- Hard gates remain green
- Evidence confidence is sufficient

BRANCHING SEARCH

When the repair is uncertain, branch the hypothesis

COMPOSITION

Candidate A

Change composition, sign placement, and negative space behind the hero.

MATERIAL / VALUE

Candidate B

Change emissive hierarchy and hero-to-background value separation.

LIGHTING

Candidate C

Change localized light placement and edge response.

ISOLATED WORKTREES

IDENTICAL REPLAY

RANDOMIZED LABELS

BOTH ORDERS

PAIRWISE CRITICS

WINNER

Selection

Keep the candidate with consistent preference on the target criterion — stable across both presentation orders — and no protected-metric regression. Preserve the prior accepted commit.

NO IMPROVEMENT

No winner

Rollback. Record the failed hypothesis. Do not merge a change merely because work was spent on it.

Current coding environments increasingly support isolated worktrees and parallel agents; use those capabilities as adapters, not as a substitute for graph policy^[5-7].

CONVERGENCE CONTROL

Stop tweaking when the abstraction is wrong

ESCALATE**Plateau trigger**

After two repair cycles without a meaningful evidence delta, stop local parameter changes and route to architecture or art-grammar review — an optional director pause (\$05).

RECURRING CLEANUP**Entropy control**

Duplicate generator audit; oversized module audit; cross-layer import audit; unbounded allocation audit; missing disposal audit; seed inconsistency audit; stale documentation audit; unowned debt audit.

Escalation questions

- Is the defect generated by the wrong system boundary?
- Is the art grammar missing a constraint?
- Is the renderer or material track incompatible?
- Is the route beat doing too many jobs?
- Is the metric measuring the wrong thing?

High-throughput agent development requires recurring cleanup and mechanically encoded architecture principles, not occasional heroic refactors^[2].

STOP CONDITION

Iteration count is a budget, not evidence

The run stops in exactly one of two recorded ways: the checklist below is fully green, or a budget expires and the run ends as a named failure with its evidence intact.

- All hard functional gates pass.
- No critical or major defect remains.
- Provenance, file, bundle, and network audits pass.
- Required screenshots, traces, logs, audio renders, and manifests exist.
- Two consecutive full regression cycles pass.
- Critic calibration is current for every model version used. **NEW**
- Deterministic replays and state hashes pass, Regime A.
- Declared p95 and p99 frame-time budgets pass.
- Required browser and device profiles pass.
- Clean checkout reproduces generated resources within regime.
- The cost ledger is complete in the run manifest. **NEW**
- The accepted build and release evidence are committed.

Budget expiry is a result, not an embarrassment

A run that exhausts tokens, wall time, or retries stops as BUDGET_EXHAUSTED with its partial evidence, cost ledger, and last accepted commit. It never stops as a quiet “good enough”.

SECTION 05 / OPERATIONS

RUNS HAVE INVOICES. HUMANS HAVE LIMITS.

v1 described a machine that could, in principle, spend without bound and answer to no one. This section prices every node, defines which gate batteries run when, and gives the one human in the loop real authority — and real constraints.

Runs have invoices

NODE	MODEL TIER	RATIONALE
Brief compile	frontier	Runs once; ambiguity here is the most expensive defect in the run.
Constitution audit	small	Mostly mechanical checks with light language verification.
Architecture decision	frontier	Highest-leverage irreversible decision; runs once per record.
Builders	mid, code	Bounded packets with executable acceptance; retries are cheap.
Structural lints + scans	none	Deterministic code. A model here is waste and a trust leak.
Visual critic	frontier, multimodal	Judgment quality is the product; calibrated per \$04.
Gameplay critic	mid, multimodal	Reads replays and traces against explicit contracts.
Performance critic	small	The math is in the trace; the model only names the dominant cost.
Provenance auditor	none	Pass or fail from scanners and manifests. No opinions involved.
Orchestrator	mid	Recommends transitions; deterministic code decides them.

The battery ladder

Smoke, every commit: build, lints, unit, one replay, provenance scans. Zero model calls.

Milestone: all eleven gates plus the calibrated critic panel.

Release: the milestone battery across the full browser matrix, twice, back to back.

Ledger rule

Every run manifest records model calls, tokens, and wall time per role (Appendix B). Budget exhaustion is a named failure state, never a silent downgrade of standards.

A critic call is an expenditure. Spend it only on builds that already pass the mechanical gates. A frontier model reviewing a build that fails lint is the most expensive way ever devised to read a compiler error.

The director holds the constitution

Exactly one human role exists inside the graph: the director. The director owns the authority documents — and is bound by the same evidence discipline as every agent.

What only the director may do

Amend CONSTITUTION.md, PRODUCT_CONTRACT.md, ART_BIBLE.md, or QUALITY_GATES.md. Every amendment lands as a DECISIONS.md entry, bumps authorityVersion, and triggers a constitution re-audit. Gates are never waived by anyone — a wrong gate is amended in the open, not skipped in the dark.

What the director may not do

Mark a failed gate green. Edit or delete evidence. Approve a release without a run ID. Decode blind labels. Ask a builder to “just try it without the test.” The dashboard renders repository state; it accepts decisions only as recorded DECISIONS entries.

Four sign-off pauses

- 01 **Contract** — after BRIEF_COMPILE: the product contract is accepted or returned.
- 02 **Freeze** — the constitution, art bible, and gates are frozen for the run.
- 03 **Renderer record** — the architecture decision and its rejected alternative.
- 04 **Release** — the release candidate with its complete evidence run.

Plus one optional pause: a plateau escalation (\$04) may request a grammar or scope ruling. The director also joins pairwise aesthetic review — under the same blind, both-order protocol as any critic.

Dashboard specification

- Run index with graph position
- Gate matrix, green and red, per run
- Protected-metric sparklines
- Latest beauty and diagnostic captures
- Defect ledger by severity and age
- Budget burn against ledger

Resolves DOC-006. “Exceptions require a recorded authority change” now names who records it, where it appears, and what it can never cover.

SECTION 06 / PROMPT LANGUAGE

AN ADJECTIVE MUST RESOLVE INTO A TEST

The prompt survives — as a compact interface to the control plane. Ten weak phrases and their operational replacements, the authority order, and the sentence test that decides what a prompt is allowed to contain.

LEXICAL LEVERAGE

Replace aesthetic wishes with operational language

WEAK PHRASE	OPERATIONAL REPLACEMENT
Make it AAA	Produce an AAA-grade vertical slice that passes declared gameplay, visual, performance, audio, provenance, compatibility, accessibility, and reproducibility gates.
Make it cinematic	Enforce camera grammar, focal hierarchy, depth separation, motion limits, and authored presentation beats.
Add detail	Allocate macro, medium, and micro detail according to the art grammar without damaging silhouette or route readability.
Make materials realistic	Generate bounded base-color, roughness, normal, emissive, and causal wear variation that responds distinctly under test lighting.
Make it dynamic	Maintain declared motion, event cadence, and causal feedback outside special states.
Optimize it	Meet profile-specific p95 and p99 frame-time budgets without deleting protected features.
Use procedural generation	Use seeded grammars, bounded distributions, rejection tests, and reproducible manifests.
Improve the scene	Resolve the named defect using the stated hypothesis, protected metrics, and acceptance test.
Fix the weakest parts	Select the highest-utility root cause under dependency, confidence, regression-risk, and cost.
Polish it	Remove all release-blocking defects and pass two clean regression cycles.

PROMPT ARCHITECTURE

The prompt is a compact interface to the control plane

Authority order

- 01 CONSTITUTION.md
- 02 PRODUCT_CONTRACT.md
- 03 ART_BIBLE.md
- 04 ARCHITECTURE.md
- 05 QUALITY_GATES.md
- 06 Active task packet
- 07 Agent judgment

INVARIANT**Conflict rule**

A lower authority cannot silently override a higher authority. Record every approved exception in DECISIONS.md.

Sentence test

Keep a sentence when it defines at least one of these:

AUTHORITY

INVARIANT

INTERFACE

ARTIFACT

METRIC

TRANSITION

FAILURE RESPONSE

STOP CONDITION

Delete or relocate

Repeated motivation, vague adjectives, stale feature lists, implementation detail that belongs in architecture, and rules with no verification method.

Rendered, not copied **NEW**

Appendix A is rendered from orchestration/prompts/orchestrator.md at document build. The repository copy is canonical; the PDF is a view. A prompt that lives only in a PDF is already stale.

An adjective must resolve into an observable test. A verb must resolve into an executable action. A prohibition must name its detector.

SECTION 07 / IMPLEMENTATION PROGRAM

SHIP THE PROGRAM, NOT THE PROMISE

Build the harness first, freeze authority early, integrate through evidence, and keep provider-specific capabilities behind adapters. Then run the red-team checklist against your own run before anyone else gets the chance.

EIGHT-PHASE PROGRAM

Implementation roadmap

A condensed operator view of the §01 canonical graph: phases 01–02 cover nodes 1–3, phase 03 builds the spine nodes 8–10 run on, phases 04–06 are node 6 specialization, phase 07 is the node 11–13 loop, phase 08 is nodes 14–15. One topology, two zoom levels.

- 01 Repository constitution**
Create contracts, schemas, ownership, structural lints, provenance policy, and a clean build.
- 02 Renderer decision proof**
Representative material, post stack, stress scene, browser matrix, trace, decision record.
- 03 Simulation and QA spine**
Fixed tick, seeded streams, input recording, state hashes, deterministic capture contract.
- 04 Gameplay graybox**
Primary verb, route states, collision, camera grammar, failure and reset, authored beats.
- 05 Procedural asset compiler**
Grammar, geometry, materials, colliders, LOD, rejection tests, provenance manifest.
- 06 Presentation integration**
Lighting, TSL and post, weather, traffic, audio, event coupling, performance profiles.
- 07 Independent convergence loop**
Evidence capture, calibrated critics, defects, candidate branches, rollback, plateau escalation.
- 08 Release audit**
Cross-browser projects, provenance, clean checkout, two clean regression cycles, final manifest with cost ledger.

MODEL-AGNOSTIC ADAPTERS

Use provider capabilities without surrendering the graph

ENVIRONMENT	USEFUL CAPABILITY	ADAPTER POLICY
Codex	Parallel agents, separate threads, isolated worktrees, sandboxed repository work ^[5] .	Map candidate tasks to worktrees; graph state remains external and authoritative.
Claude Code	Custom subagents with separate context, tailored prompts, tool restrictions, and permissions ^[6] .	Use for bounded specialists; the main orchestrator performs chaining.
Gemini CLI	Specialist subagents with independent context and specialized tools ^[7] .	Parallelize research and disjoint ownership; serialize shared code edits.
OpenAI Agents SDK	Manager-as-tools, handoffs, code-controlled sequencing, parallel calls, structured output, tracing ^[4] .	Use code orchestration for deterministic transitions; let LLMs reason inside nodes.
Any shell agent	Git, worktrees, commands, files, browser automation.	Conform to task packet and return schema. Provider name never changes release predicates.

Invariant

The core system is transport-neutral. A provider adapter translates dispatch, workspace creation, permissions, tool access, and result collection into the graph’s typed interface — and reports its calls into the same cost ledger.

RED-TEAM CHECKLIST

Anti-patterns that reintroduce slop

REJECT

Prompt accumulation

Every failure adds another paragraph to one giant prompt instead of changing the graph, test, or contract.

REJECT

Agent theater

Many named agents share the same context, tools, authority, and write access, so they are not independent.

REJECT

Metric theater

Numbers are recorded without device profile, baseline, distribution, tolerance, or release predicate.

REJECT

Procedural texture noise

Noise is added everywhere to imply detail, producing dirt without material causality.

REJECT

Parallel merge chaos

Multiple agents edit shared renderer, package, and architecture surfaces concurrently.

REJECT

Critic persuasion

The builder provides a long explanation before the critic sees the image.

REJECT

Baseline laundering

A regression is accepted by updating golden screenshots without an explicit review decision.

REJECT

Budget-by-deletion

Signature systems disappear instead of being replaced by cheaper perceptual equivalents.

REJECT

NEW

Calibration skip

Critic findings are admitted from a model, version, or prompt that has no current calibration pass — verdicts with no error bars.

REJECT

NEW

Regime laundering

A Regime A artifact is quietly re-filed as Regime B, or a tolerance is widened in a task packet, to make a failing comparison pass.

RELEASE CANDIDATE CHECKLIST

Twenty-two boxes, zero adjectives

- Scope matches PRODUCT_CONTRACT.md.
- Authored beats are present and replayable.
- Materials obey the art grammar and causal wear rules.
- All named profiles pass p95 and p99 budgets.
- Replay hashes match accepted baselines, Regime A.
- Two consecutive full regression runs pass.
- No forbidden media or opaque blob exists — in src or the built bundle.
- Accepted commit, authority digest, and evidence index are recorded.
- Audio loudness and true peak meet declared targets^[21].
NEW
- Voice budget holds; zero dropout or clipping events.
NEW
- Critic calibration is current for every configuration used.
NEW
- Primary verb, completion, failure, reset, and recovery pass.
- Hero silhouette and route read at thumbnail and grayscale.
- Diagnostic views show controlled emissive, depth, and motion.
- No shader compilation or allocation blocker remains.
- All release artifacts have complete identity.
- Network audit contains no unapproved request.
- Clean checkout regenerates declared resources within regime.
- Limitations and deferred issues are explicit.
- Reduced-motion run passes; CVD route captures pass^[22,23]. **NEW**
- HUD contrast report meets 4.5:1. **NEW**
- Cost ledger is complete in the run manifest. **NEW**

Every box maps to a gate in \$00 and an artifact class in \$02. A checklist item with no producing command is decoration; none of these are decoration.

FINAL OPERATING PRINCIPLES

What the system must make impossible

01

Unsupported completion

No evidence path means no completion claim.

02

Invisible cheating

No resource reaches runtime without source, seed, manifest, scan, and network audit.

03

Unbounded parallelism

No two workers edit shared integration surfaces without serialized ownership.

04

Self-certified quality

No builder approves its own visual, performance, provenance, or release result.

05

Randomness as art direction

No generator operates without vocabulary, ranges, adjacency, rejection, and composition constraints.

06

Performance by erasure

No protected identity disappears merely to hit a number.

07

Endless local tweaking

Plateau forces architecture review, candidate branching, or explicit stop.

08

Conversation-only memory

Every enduring decision is repository-local, versioned, and mechanically discoverable.

09

NEW

Uncalibrated judgment

No critic verdict is admissible without a current calibration pass for that exact model, version, and prompt.

10

NEW

Silent authority

No human override exists outside a recorded decision. The director acts through DECISIONS.md or not at all.

The decisive leverage is not a longer request. It is a system in which weak work cannot hide behind fluent language.

APPENDIX A / DEPLOYABLE SYSTEM PROMPT

Three.js Evidence Graph Orchestrator — 1/4

Rendered at document build from orchestration/prompts/orchestrator.md. The repository copy is canonical; this appendix is a view of it. Blocks changed in v2 are marked // v2.

SYSTEM PROMPT: THREE.JS EVIDENCE GRAPH ORCHESTRATOR

ROLE

You are the orchestration controller for an evidence-driven Three.js game production graph. You do not treat prose, implementation effort, code presence, self-assigned scores, or builder confidence as proof. Only reproducible artifacts produced by the verification system constitute evidence. Your job is to coordinate specialized workers, maintain typed project state, enforce repository laws, select graph transitions, preserve accepted quality, reject unsupported claims, and produce one evidence-backed browser vertical slice.

MISSION

Build the vertical slice defined in PRODUCT_CONTRACT.md. The target is an AAA-grade vertical slice, not an entire open-world game. Quality means that the declared gameplay, visual, temporal, performance, audio, compatibility, accessibility, provenance, and reproducibility gates pass on the declared device profiles. // v2: audio, accessibility Do not expand scope beyond the product contract. New ideas that do not repair an accepted defect must be recorded as deferred issues rather than added to the active milestone.

AUTHORITY ORDER

Resolve conflicts using this order:
 1. CONSTITUTION.md 2. PRODUCT_CONTRACT.md 3. ART_BIBLE.md 4. ARCHITECTURE.md
 5. QUALITY_GATES.md 6. Active task packet 7. Agent judgment
 A lower authority may not silently override a higher authority. Record every approved exception in DECISIONS.md.

TRUST MODEL

Builder output is untrusted until independently verified.
 A builder may not approve its own work.
 A critic may not edit implementation code.
 A critic's findings are inadmissible without a current calibration pass. // v2
 An orchestrator may not replace failed evidence with its own opinion.
 A performance worker may not delete protected visual or gameplay features.
 An integration agent may not merge a candidate that regresses a hard gate.
 All completion claims must cite run IDs, artifact paths, metric files, test results, and commit hashes.

TECHNOLOGY POLICY

Pin the exact Three.js release.
 Before full implementation, run a renderer decision proof comparing:
 A. WebGPURenderer with TSL and its WebGL 2 fallback.
 B. WebGLRenderer when pure WebGL 2 stability or compatibility requires it.
 Record the selected renderer and rejected alternative in ARCHITECTURE.md. Do not maintain two complete rendering pipelines without an explicit product requirement.
 Use Three.js and approved browser APIs for runtime rendering and game systems. Use approved development tools for builds, tests, browser automation, profiling, and static verification.

ASSET PROVENANCE AND SUPPLY CHAIN LAW

Do not download, import, embed, encode, or retrieve external models, textures, HDRIs, fonts, sprites, animation files, stock imagery, prerecorded audio, or asset packs.
 Every visible and audible artifact must be generated by deterministic, source-controlled code from explicit parameters and seeds.
 Every generator must produce a provenance record containing its source version, seed, parameters, output hash, and regeneration command.
 The browser must make no unapproved network request after the local application bundle loads.
 The dependency graph is an asset channel: installs are lockfile-pinned and build-time only; // v2
 packages that ship media files are forbidden or stripped at build; the provenance scanner runs on the built bundle, not only on source; the runtime origin allowlist is empty beyond the local bundle.
 UI text uses a pinned per-profile system font stack or generated glyph atlases; no font files. // v2
 Fail the run if forbidden media, opaque binary additions, oversized encoded blobs, unmatched generated outputs, or unexpected network requests are found.

APPENDIX A / DEPLOYABLE SYSTEM PROMPT

Three.js Evidence Graph Orchestrator — 2/4

REPOSITORY CONTROL PLANE

Use the repository as the system of record. AGENTS.md is a short map, not an encyclopedia. Maintain CONSTITUTION.md, PRODUCT_CONTRACT.md, ART_BIBLE.md, ARCHITECTURE.md, QUALITY_GATES.md, WORKFLOW.md, DECISIONS.md, PROVENANCE.json, docs/exec-plans, tasks, evidence, orchestration, orchestration/prompts, tools/scanners, tests, and tests/critics/calibration. // v2 paths
Do not rely on information that exists only in conversation history.

GRAPH EXECUTION

Execute this graph:

- | | |
|----------------------------------|--|
| 1. BRIEF_COMPILE | 9. DETERMINISTIC_REPLAY |
| 2. CONSTITUTION_AUDIT | 10. EVIDENCE_CAPTURE |
| 3. ARCHITECTURE_DECISION | 11. INDEPENDENT_CRITICISM |
| 4. ART_AND_GAMEPLAY_BIBLE_FREEZE | 12. EVIDENCE_REDUCTION |
| 5. MILESTONE_PLAN | 13. ACCEPT, REPAIR, ROLLBACK, OR ARCHITECTURE_REVIEW |
| 6. SPECIALIST_IMPLEMENTATION | 14. CROSS_BROWSER_RELEASE_AUDIT |
| 7. INTEGRATION | 15. RELEASE_CANDIDATE |
| 8. STATIC_VERIFICATION | |

Deterministic code controls graph transitions. An LLM may recommend a transition but may not bypass the predicate.

Persist typed GraphState, including the cost ledger, after every node. // v2

SUBAGENT POLICY

Spawn only specialists required by the active node.

Use isolated worktrees for competing candidates.

Parallel work is permitted only when ownership is disjoint or work is read-only. Serialize edits to shared architecture, package configuration, global renderer setup, and integration surfaces.

Every worker receives a bounded task packet containing task ID, objective, context paths, allowed files, forbidden files, invariants, acceptance commands, expected evidence, resource budget with model tier, and failure statuses. Packets validate against task-packet.schema.json. // v2

Every worker returns status, changed files, commit hash, commands executed, evidence paths, unresolved risks, and discovered follow-up issues.

A prose-only completion response is invalid.

GAME ARCHITECTURE

Use a fixed authoritative simulation tick and render interpolation.

Do not use Math.random for authoritative generation or gameplay. Use named deterministic random streams per system.

When cross-engine replay identity is gated, authoritative simulation must use the repository's deterministic math kernel instead of bare Math transcendental functions. // v2

Record input by simulation tick. Generate periodic deterministic state hashes. Make every required gameplay state replayable without manual input.

Use explicit state machines for route, boost, crash, reset, completion, and other product-critical states.

Prefer bounded arcade simulation over unnecessary general-purpose systems.

EVIDENCE REGIMES

// v2 block

Every artifact class declares Regime A (bit-exact) or Regime B (declared tolerance) in QUALITY_GATES.md.

Regime A: simulation state hashes, input replays, seed manifests, generator geometry and data buffers, offline audio PCM on the pinned profile. Comparator: content hash. Any drift fails.

Regime B: all GPU-rasterized output and all cross-profile comparisons. Comparator: perceptual diff within the per-view, per-profile tolerance recorded in QUALITY_GATES.md.

Tolerances are read only from QUALITY_GATES.md, never from a task packet. Reclassifying an artifact's regime or widening a tolerance requires a recorded DECISIONS.md entry.

PROCEDURAL ART SYSTEM

Procedural generation must follow ART_BIBLE.md.

Each generator must define invariant design features, variation dimensions, permitted ranges, attachment and adjacency rules, material families, LOD policy, collision proxy policy, rejection conditions, seed handling, and provenance output.

Do not use randomness as composition.

Author the presentation route as deliberate visual and gameplay beats. Generated variation may fill a beat but may not replace its intended function.

APPENDIX A / DEPLOYABLE SYSTEM PROMPT

Three.js Evidence Graph Orchestrator — 3/4

QUALITY AND EVIDENCE

For every accepted milestone, produce build/static results, deterministic replay, state-hash result, console/device log, beauty screenshots, diagnostic screenshots, frame-time trace, renderer statistics, memory statistics, network log, provenance audit, gameplay event trace, offline audio render with loudness and true-peak report, commit hash, and defect ledger. // v2: audio
Screenshot identity must include scene, seed, simulation tick, viewport, quality profile, renderer backend, math kernel version, UI state, and commit. // v2: math kernel
Render the authoritative audio event timeline with OfflineAudioContext. Gate integrated loudness within the declared target, true peak at or below the declared ceiling, concurrent voices within budget, zero dropout or hard-clip events, and event-to-audio offsets within declared bounds. // v2
Capture reduced-motion runs, CVD-simulated route views, and a HUD contrast report for the accessibility gate. // v2
Do not claim a feature is successful because it exists in code. If it is not visible, measurable, or behaviorally verified, it has not passed.

CRITIC POLICY

Run separate read-only critics for visual composition/art direction, gameplay/camera, performance/frame pacing, audio mix, asset provenance/reproducibility, and architecture/regression risk. // v2: audio critic
Visual critics receive artifacts and the art bible, not builder explanations. Performance critics receive traces and budgets, not aesthetic arguments.
Every criticism must cite an artifact and identify an observable defect. Reject generic feedback such as add polish, make it more cinematic, or improve the lighting.

CRITIC CALIBRATION

// v2 block

Before admitting findings from any critic configuration (model, version, prompt), run it against tests/critics/calibration: evidence pairs with known, seeded defects and known clean baselines. Admit the configuration only at or above the declared calibration accuracy in QUALITY_GATES.md. Re-run calibration on any model, version, or prompt change. Record the calibration run ID in the manifest of every run that uses the configuration.
Run every pairwise comparison in both presentation orders. Count a preference only when stable across orders; record an unstable preference as a tie.
Report inter-critic agreement per criterion. Route persistent single-criterion disagreement to art-bible review; never resolve it by averaging.
A critic without a current calibration pass has opinions, not findings.

REPAIR POLICY

Do not automatically fix two low numerical scores.
Create a defect record containing observed failure, evidence, root-cause hypothesis, proposed intervention, expected evidence delta, protected metrics, acceptance test, and rollback condition. Defect records validate against defect-record.schema.json. // v2
Prioritize the defect with the highest expected quality gain after dependencies, confidence, regression risk, and cost.
When repair is uncertain, create two or three isolated candidates. Capture them under identical conditions and use blind pairwise evaluation in both orders.
Accept a candidate only when it improves the target criterion without regressing hard gates or protected metrics. Roll back failed candidates.

APPENDIX A / DEPLOYABLE SYSTEM PROMPT

Three.js Evidence Graph Orchestrator — 4/4

PERFORMANCE POLICY

Measure before modifying.

Record median, p95, and p99 frame time, 1 percent low, long-frame count, simulation time, GPU time where available, draw calls, triangles, geometry count, texture count, active effects, memory growth, and shader compilation stalls.

Optimize the dominant measured cost first. Prefer InstancedMesh, BatchedMesh, pooling, reuse, atlases, culling, LOD, segment recycling, lower-resolution post passes, shader precompilation, bounded transparency, and adaptive quality.

Do not delete the defining art or gameplay identity to satisfy a budget. Replace an expensive mechanism with a cheaper perceptual equivalent.

ECONOMY

// v2 block

Route each node to its declared model tier from QUALITY_GATES.md; never route mechanical checks to a model at all.

Record every model call in the cost ledger: role, model, calls, tokens, wall time. The ledger is part of the run manifest and validates against run-manifest.schema.json.

Dispatch critics only on builds that already pass the mechanical smoke battery.

Budget exhaustion ends the run as BUDGET_EXHAUSTED with partial evidence, the ledger, and the last accepted commit. Never lower a gate to fit a budget.

DIRECTOR AND AUTHORITY CHANGES

// v2 block

Exactly one human role, the director, may amend CONSTITUTION.md, PRODUCT_CONTRACT.md, ART_BIBLE.md, or QUALITY_GATES.md. Every amendment is a DECISIONS.md entry that bumps authorityVersion and triggers a constitution re-audit.

Pause for director sign-off at: product contract acceptance, authority freeze, renderer decision record, and release candidate. A plateau escalation may request an optional grammar or scope ruling.

No one, including the director, may mark a failed gate green, edit evidence, approve a release without a run ID, or decode blind labels. Directors participate in pairwise aesthetic review under the same blind, both-order protocol as any critic.

CONVERGENCE AND STOP CONDITIONS

Iteration count alone is not evidence.

Release only when all hard gates pass, deterministic replays pass under their declared regimes, no critical or major defect remains, frame-time budgets pass, audio and accessibility gates pass, provenance, bundle, and network audits pass, required browser/device profiles pass, required artifacts exist, a clean checkout reproduces resources, two consecutive regression cycles pass, critic calibration is current, the cost ledger is complete, and the accepted build is committed. After two repair cycles without meaningful improvement, stop local tweaking and route to ARCHITECTURE_REVIEW.

When a hard resource limit is reached before convergence, preserve the best verified build and return an incomplete report containing failed gates, evidence, attempted repairs, the cost ledger, and recommended next graph node. Do not claim completion.

FINAL REPORT

Return product and renderer decisions, install/run instructions, controls, deterministic replay commands, evidence index, performance profiles, audio report, accessibility report, provenance audit, browser/device matrix, cost ledger, accepted commit hash, remaining limitations, deferred issues, and exact failed gates if any.

Do not use the word AAA in the final report unless every declared release gate has passed.

Validation is code, not etiquette — 1/2

JSON Schema draft-07. These are the normative contracts referenced in §01 and §04; the repository copies under `orchestration/` are authoritative. A packet, defect record, or manifest that fails validation is rejected by code before any model reads it.

```
// task-packet.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "task-packet.schema.json",
  "title": "TaskPacket",
  "type": "object",
  "additionalProperties": false,
  "required": ["task_id", "objective", "context_paths",
    "allowed_files", "forbidden_files", "invariants",
    "acceptance_commands", "expected_evidence",
    "budget", "failure_statuses"],
  "properties": {
    "task_id": {
      "type": "string",
      "pattern": "^[A-Z]{2,4}-[0-9]{3,}$"
    },
    "objective": {
      "type": "string", "minLength": 12,
      "maxLength": 240
    },
    "context_paths": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "allowed_files": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "forbidden_files": {
      "type": "array",
      "items": { "type": "string" }
    },
    "invariants": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "acceptance_commands": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "expected_evidence": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "budget": {
      "type": "object",
      "additionalProperties": false,
      "required": ["wall_time_min", "retries",
        "model_tier"],
      "properties": {
        "wall_time_min": { "type": "integer",
          "minimum": 1 },
        "retries": { "type": "integer",
          "minimum": 0 },
        "model_tier": { "enum": ["none", "small",
          "mid", "frontier"] }
      }
    },
    "failure_statuses": {
      "type": "array", "minItems": 1,
      "items": { "enum": ["BLOCKED",
        "NO_IMPROVEMENT", "REGRESSION",
        "BUDGET_EXHAUSTED"] }
    }
  }
}
```

```
// defect-record.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "defect-record.schema.json",
  "title": "DefectRecord",
  "type": "object",
  "additionalProperties": false,
  "required": ["issue_id", "evidence",
    "observed_failure", "root_cause_hypothesis",
    "proposed_intervention", "expected_delta",
    "protected_metrics", "acceptance_test",
    "rollback_condition"],
  "properties": {
    "issue_id": {
      "type": "string",
      "pattern": "^[A-Z]{3}-[0-9]{3,}$"
    },
    "evidence": {
      "type": "array", "minItems": 1,
      "items": { "type": "string",
        "pattern": "^run-[0-9]+/" }
    },
    "observed_failure": {
      "type": "string", "minLength": 20
    },
    "root_cause_hypothesis": {
      "type": "string", "minLength": 20
    },
    "proposed_intervention": {
      "type": "string", "minLength": 20
    },
    "expected_delta": {
      "type": "string", "minLength": 12
    },
    "protected_metrics": {
      "type": "array", "minItems": 1,
      "items": { "type": "string" }
    },
    "acceptance_test": {
      "type": "string",
      "pattern": "\\spec\\. [a-z]+$"
    },
    "rollback_condition": {
      "type": "string", "minLength": 12
    },
    "severity": {
      "enum": ["critical", "major", "minor"]
    },
    "critic_id": { "type": "string" },
    "calibration_run": { "type": "string" }
  }
}

// severity, critic_id, and calibration_run are
// attached by the critic pipeline; the nine
// required fields are the minimum a repair
// transition will accept (see §04). Generic
// feedback cannot satisfy observed_failure +
// evidence + acceptance_test, which is the
// point.
```

APPENDIX B / NORMATIVE SCHEMAS

Validation is code, not etiquette — 2/2

```
// run-manifest.schema.json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "run-manifest.schema.json",
  "title": "RunManifest",
  "type": "object",
  "additionalProperties": false,
  "required": ["run_id", "commit",
    "authority_version", "authority_digest",
    "seed", "profile", "gates", "artifacts",
    "costLedger"],
  "properties": {
    "run_id": {
      "type": "string",
      "pattern": "^run-[0-9]{4,}$"
    },
    "commit": {
      "type": "string",
      "pattern": "^[0-9a-f]{7,40}$"
    },
    "authority_version": { "type": "string" },
    "authority_digest": {
      "type": "string",
      "pattern": "^sha256:[0-9a-f]{64}$"
    },
    "seed": { "type": "integer", "minimum": 0 },
    "profile": {
      "type": "object",
      "additionalProperties": false,
      "required": ["browser", "viewport",
        "renderer", "quality", "mathkernel"],
      "properties": {
        "browser": { "type": "string" },
        "viewport": { "type": "string",
          "pattern": "^[0-9]+x[0-9]+$" },
        "renderer": { "enum": ["webgpu",
          "webgl2"] },
        "quality": { "enum": ["high", "medium",
          "low"] },
        "mathkernel": { "type": "string" }
      }
    },
    "gates": {
      "type": "object",
      "additionalProperties": {
        "type": "object",
        "required": ["status", "regime"],
        "properties": {
          "status": { "enum": ["pass", "fail",
            "skipped"] },
          "regime": { "enum": ["A", "B", "n/a"] },
          "evidence": { "type": "array",
            "items": { "type": "string" } }
        }
      }
    }
  },
}
```

```
"artifacts": {
  "type": "array", "minItems": 1,
  "items": {
    "type": "object",
    "required": ["path", "hash", "regime"],
    "properties": {
      "path": { "type": "string" },
      "hash": { "type": "string" },
      "regime": { "enum": ["A", "B"] },
      "tolerance_ref": { "type": "string" }
    }
  }
},
"costLedger": {
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "required": ["role", "model", "calls",
      "tokens", "wallTimeMs"],
    "properties": {
      "role": { "type": "string" },
      "model": { "type": "string" },
      "calls": { "type": "integer",
        "minimum": 0 },
      "tokens": { "type": "integer",
        "minimum": 0 },
      "wallTimeMs": { "type": "integer",
        "minimum": 0 }
    }
  }
},
"calibration_runs": {
  "type": "object",
  "additionalProperties": {
    "type": "string" }
},
"status": {
  "enum": ["accepted", "rejected",
    "BUDGET_EXHAUSTED"]
}
}
```

Why costLedger is required

A manifest without an invoice hides the run's real shape. An empty array is valid — a zero-model smoke run — but an absent field fails validation, so “we forgot to count” is not an expressible state.

Enforcement point

Schemas run in the smoke battery (\$05): every packet before dispatch, every defect record before a repair transition, every manifest at evidence capture. Structural lint, zero model calls.

PRIMARY SOURCES

Research references

Current API and product facts are grounded in official documentation.

Research claims use primary papers. The architecture and wording system are engineering synthesis developed for this edition.

- [1] Supplied source: Neon Monsoon Courier — Iteration-Driven Three.js Implementation Playbook v7. User-provided PDF, 21 pages.
- [2] OpenAI. Harness engineering: leveraging Codex in an agent-first world. 11 Feb 2026. <https://openai.com/index/harness-engineering/>
- [3] OpenAI. An open-source spec for Codex orchestration: Symphony. 27 Apr 2026. <https://openai.com/index/open-source-codex-orchestration-symphony/>
- [4] OpenAI Agents SDK. Agent orchestration. https://openai.github.io/openai-agents-python/multi_agent/
- [5] OpenAI. Introducing the Codex app. 2 Feb 2026, updated 4 Mar 2026. <https://openai.com/index/introducing-the-codex-app/>
- [6] Anthropic. Claude Code docs: Create custom subagents. <https://code.claude.com/docs/en/sub-agents>
- [7] Google. Gemini CLI docs: Subagents. <https://geminicli.com/docs/core/subagents/>
- [8] Three.js release repository. r185 listed as latest, 1 Jul 2026. <https://github.com/mrdoob/three.js/releases>
- [9] Three.js Docs. WebGPURenderer. <https://threejs.org/docs/pages/WebGPURenderer.html>
- [10] Three.js Wiki. TSL Spec. <https://github.com/mrdoob/three.js/wiki>
- [11] Three.js Docs. Timer and Clock. <https://threejs.org/docs/pages/Timer.html> and <https://threejs.org/docs/pages/Clock.html>
- [12] Three.js Docs. InstancedMesh and BatchedMesh. <https://threejs.org/docs/pages/InstancedMesh.html> and <https://threejs.org/docs/pages/BatchedMesh.html>
- [13] Three.js Docs. Renderer and Info, including compileAsync and renderer statistics. <https://threejs.org/docs/pages/Renderer.html>
- [14] Three.js Docs. Octree. <https://threejs.org/docs/pages/Octree.html>
- [15] Microsoft Playwright. Visual comparisons. <https://playwright.dev/docs/test-snapshots>
- [16] Microsoft Playwright. Projects. <https://playwright.dev/docs/test-projects>
- [17] MDN Web Docs. Web Audio API, AudioWorklet, and PannerNode. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- [18] Huang, J. et al. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798, ICLR 2024.
- [19] Zhuge, M. et al. Language Agents as Optimizable Graphs. arXiv: 2402.16823, 2024.
- [20] Ecma International. ECMA-262, ECMAScript Language Specification — function properties of the Math object are implementation-approximated. <https://tc39.es/ecma262/> **NEW**
- [21] ITU-R. Recommendation BS.1770-4: Algorithms to measure audio programme loudness and true-peak audio level. <https://www.itu.int/rec/R-REC-BS.1770> **NEW**
- [22] W3C. Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/> **NEW**
- [23] MDN Web Docs. prefers-reduced-motion media feature. <https://developer.mozilla.org/en-US/docs/Web/CSS/@media/prefers-reduced-motion> **NEW**
- [24] Zheng, L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena — documents position bias in pairwise model judging. arXiv:2306.05685, NeurIPS 2023. **NEW**

Document status

This is a production framework and research-backed design specification, applied to itself through the \$00 defect ledger. It is **not** a benchmark result for a completed game. **No benchmark run of this system exists yet**; fig. 0 on the cover is an illustrative target profile, not run data, and is labeled as such.

The schemas in Appendix B are normative and shipped in full. The remaining executable surface — orchestration code, scanners, calibration fixtures — is specified here and published as a companion repository; that publication, with one complete evidence run, is the release criterion for v2.x. Performance budgets must be calibrated on declared target devices before any release claim. Until an evidence run exists, this document claims design validity, not results.